

PRODUCT AND CATEGORY MANAGEMENT SYSTEM

Scenario

The e-commerce database needs to organize product information according to different categories. The system should maintain the details of each product and establish a proper connection between products and their respective categories. This makes it easier to manage products, update their information, and generate useful category-based reports.

1. Design Product and Category Tables

Category Table:

The Category table is designed to maintain the categories available in the e-commerce system. Each category is assigned a unique identifier so that it can be distinguished from other categories.

```
SQL> CREATE TABLE CATEGORY (CATEGORY_ID VARCHAR(7), CATEGORY_NAME VARCHAR(20), DESCRIPTION VARCHAR(50));  
Table created.
```

Product Table:

The Product table stores the details of products available on the platform. It maintains important information required for identifying and managing each product.

```
SQL> CREATE TABLE PRODUCT (PRODUCT_ID VARCHAR(7), PRODUCT_NAME VARCHAR(28), CATEGORY_ID VARCHAR(7), PRICE NUMBER(9,2), STOCK_QUANTITY NUMBER);  
Table created.
```

The Category and Product tables are related because every product is associated with a particular category.

2. Define Relationships and Constraints

Relationships:

The relationship between the Category and Product tables is **One-to-Many (1 : *)**.

This means that one category can be associated with multiple products, whereas a product belongs to one category.

Primary Key: A Primary Key provides a unique identity to every record in a table. It prevents two records from having the same identifying value.

In the Category table, CATEGORY_ID is used to identify each category. In the Product table, PRODUCT_ID uniquely identifies each product.

```
SQL> ALTER TABLE Category ADD CONSTRAINT PK_Category PRIMARY KEY (Category_ID);  
Table altered.  
  
SQL> ALTER TABLE Product ADD CONSTRAINT PK_Product PRIMARY KEY (Product_ID);  
Table altered.
```

Foreign Key: A Foreign Key is used to establish a connection between related tables. In the Product table, CATEGORY_ID connects each product with the appropriate record in the Category table.

```
SQL> ALTER TABLE Product ADD CONSTRAINT FK_Product_Category FOREIGN KEY (Category_ID) REFERENCES Category (Category_ID);  
Table altered.
```

NOT NULL Constraint: The NOT NULL constraint ensures that essential columns cannot be left without a value. This is useful for fields where information is compulsory, such as product or category names.

```
SQL> ALTER TABLE Category MODIFY Category_Name VARCHAR(100) NOT NULL;  
Table altered.  
  
SQL> ALTER TABLE Product MODIFY Product_Name VARCHAR(150) NOT NULL;  
Table altered.
```

CHECK Constraint: The CHECK constraint is used to restrict the values entered into a column according to a specified condition. It helps maintain valid data in the database.

For example, a product price can be restricted to positive values.

```
SQL> ALTER TABLE Product ADD CONSTRAINT CHK_Price CHECK (Price > 0);  
Table altered.  
  
SQL> ALTER TABLE Product ADD CONSTRAINT CHK_Stock CHECK (Stock_Quantity >= 0);  
Table altered.
```

UNIQUE Constraint: The UNIQUE constraint prevents duplicate values from being stored in a column. It is useful when a particular value should occur only once in the table.

For example, category names can be maintained without duplicate entries.

```
SQL> ALTER TABLE Category ADD CONSTRAINT UQ_Category UNIQUE (Category_Name);  
Table altered.
```

3. Insert Values

After creating the Category and Product tables and applying the required constraints, records are inserted into the tables.

The INSERT statement is used to add new category and product information to the database. This provides the initial data required for further operations and analysis.

Category Table:

```

SQL> INSERT INTO category VALUES ('CAT001','Organic Food','Natural organic food products');
1 row created.

SQL> INSERT INTO category VALUES ('CAT002','Personal Care','Natural personal care items');
1 row created.

SQL> INSERT INTO category VALUES ('CAT003','Bamboo Products','Eco friendly bamboo products');
1 row created.

SQL> INSERT INTO category VALUES ('CAT004','Kitchen Items','Reusable kitchen products');
1 row created.

SQL> INSERT INTO category VALUES ('CAT005','Eco Stationery','Environment friendly stationery');
1 row created.

```

Product Table:

```

SQL> INSERT INTO product VALUES ('PR0001','Organic Oats','CAT001',200.00,45);
1 row created.

SQL> INSERT INTO product VALUES ('PR0002','Natural Honey','CAT001',320.00,35);
1 row created.

SQL> INSERT INTO product VALUES ('PR0003','Aloe Face Cream','CAT002',250.00,40);
1 row created.

SQL> INSERT INTO product VALUES ('PR0004','Bamboo Comb','CAT003',100.00,90);
1 row created.

SQL> INSERT INTO product VALUES ('PR0005','Bamboo Tray','CAT003',400.00,30);
1 row created.

```

After inserting the records, the contents of both tables can be displayed to verify that the data has been stored successfully.

```
SQL> SELECT * FROM Category;
```

CATEGORY_ID	CATEGORY_NAME	DESCRIPTION
CAT001	Organic Food	Natural organic food products
CAT002	Personal Care	Natural personal care items
CAT003	Bamboo Products	Eco friendly bamboo products
CAT004	Kitchen Items	Reusable kitchen products
CAT005	Eco Stationery	Environment friendly stationery

```
SQL> SELECT * FROM Product;
```

PRODUCT_ID	PRODUCT_NAME	CATEGORY_ID	PRICE	STOCK_QUANTITY
PR0001	Organic Oats	CAT001	200.00	45
PR0002	Natural Honey	CAT001	320.00	35
PR0003	Aloe Face Cream	CAT002	250.00	40
PR0004	Bamboo Comb	CAT003	100.00	90
PR0005	Bamboo Tray	CAT003	400.00	30

4. CRUD Operations

CRUD refers to the basic operations performed on database records: **Create, Read, Update, and Delete**. These operations allow the database administrator or application to manage product information throughout its lifecycle.

Insert Record: The Insert operation is used to add a new product record to the Product table. It allows newly available products to be included in the database.

```
SQL> INSERT INTO product VALUES ('PRO011','Neem Shampoo','CAT002',280.00,50);  
1 row created.
```

Update Price: The Update operation is used to modify existing product information. In this case, it can be used to change the price of a product when its selling price is revised.

```
SQL> UPDATE product SET price = 300.00 WHERE product_id = 'PRO001';  
1 row updated.
```

Delete a Product: The Delete operation removes an unwanted product record from the Product table. It can be used when a product is no longer available or needs to be removed from the database.

```
SQL> DELETE FROM product WHERE product_id = 'PRO011';  
1 row deleted.
```

5. Generate Category-wise Product Reports

- **Display all products under each category:**

This query is used to display the products along with their corresponding categories. It provides a clear view of which products are associated with each category.

```
SQL> SELECT c.Category_Name, p.Product_Name, p.price, p.Stock_Quantity FROM Product p JOIN Category c ON p.Category_ID = c.Category_ID;  
CATEGORY_NAME  PRODUCT_NAME  PRICE STOCK_QUANTITY  
-----  
Organic Food    Organic Oats   300.00 45  
Organic Food    Natural Honey  320.00 35  
Personal Care   Aloe Face Cream 250.00 48  
Bamboo Products Bamboo Comb    100.00 98  
Bamboo Products Bamboo Tray    480.00 38  
Kitchen Items   Steel Lunch Box 600.00 55  
Eco Stationery   Eco Diary       140.00 70  
Cloth Bags       Cotton Tote Bag 150.00 65  
Sustainable Fashion Eco Cotton Kurta 850.00 25  
Farming Products Organic Seed Pack 110.00 95  
10 rows selected.
```

- **Count number of products in each category:**

This query determines the number of products available in every category. The result helps identify categories that contain a larger or smaller number of products.

```
SQL> SELECT c.Category_Name, COUNT(p.Product_ID) AS Product_Count FROM Product p JOIN Category c ON p.Category_ID = c.Category_ID GROUP BY c.Category_Name;  
CATEGORY_NAME  PRODUCT_COUNT  
-----  
Sustainable Fashion 1  
Organic Food         2  
Bamboo Products      2  
Kitchen Items        1  
Personal Care        1  
Eco Stationery       1  
Cloth Bags           1  
Farming Products     1  
8 rows selected.
```

- **Find highest-priced product in each category:**

This query identifies the product with the highest price within each category. It can be used to compare the maximum-priced products across different categories.

```
SQL> SELECT c.Category_Name, p.Product_Name, p.Price AS Highest_Price FROM Product p JOIN Category c ON p.Category_ID = c.Category_ID WHERE p.Price = (SELECT MAX(p2.Price) FROM Product p2 WHERE p2.Category_ID = p.Category_ID);
```

CATEGORY_NAME	PRODUCT_NAME	HIGHEST_PRICE
Organic Food	Natural Honey	320
Personal Care	Aloe Face Cream	250
Bamboo Products	Bamboo Tray	400
Kitchen Items	Steel Lunch Box	600
Eco Stationery	Eco Diary	140
Cloth Bags	Cotton Tote Bag	100
Sustainable Fashion	Eco Cotton Kurta	850
Farming Products	Organic Seed Pack	110

8 rows selected.

- **Display categories having more than 5 products:**

This query displays only those categories whose product count is greater than two. It helps identify categories with a relatively large collection of products.

```
SQL> SELECT c.Category_Name FROM Product p JOIN Category c ON p.Category_ID = c.Category_ID GROUP BY c.Category_Name HAVING COUNT(p.Product_ID) > 2;
```

no rows selected

- **Find average product price category-wise:**

This query calculates the average price of products belonging to each category. The result gives an overview of the general price range of products within each category.

```
SQL> SELECT c.Category_Name, AVG(p.Price) AS Avg_Price FROM Product p JOIN Category c ON p.Category_ID = c.Category_ID GROUP BY c.Category_Name;
```

CATEGORY_NAME	AVG_PRICE
Sustainable Fashion	850
Organic Food	310
Bamboo Products	250
Kitchen Items	600
Personal Care	250
Eco Stationery	140
Cloth Bags	100
Farming Products	110

8 rows selected.

Conclusion:

The Product and Category Management System provides a structured way to organize ecommerce product information. By establishing relationships and applying appropriate constraints, the database maintains data consistency. CRUD operations make it possible to manage product records, while category-wise SQL queries provide useful information for analyzing products and their prices.